PROGRAMACIÓN EN MÓVILES

CINE CASTILLO

Guía de sustentación

Laboratorio 12 · requisitos originales del docente

Preparada para explicar, demostrar y responder

Una guía en lenguaje claro para comprender cómo funciona el proyecto.

Estudiante

Jesús Castillo Sumire

Sección

C24D

Curso

Programación en Móviles

Fecha

13 de noviembre de 2026

ALCANCE: SOLO LO SOLICITADO EN EL LABORATORIO

Tu sustentación en cinco minutos

La meta es explicar el recorrido de los datos, no recitar el código. Abre la aplicación y avanza por estas demostraciones en orden. El tiempo es orientativo; si el docente pregunta, prioriza explicar tus decisiones.

Tiempo	Qué mostrar	Qué decir con tus palabras
0:00–0:40	Inicio de sesión como usuario	“La app valida que los campos no estén vacíos. Firebase Authentication comprueba el correo y la contraseña; luego se consulta el rol guardado para esa cuenta.”
0:40–1:30	Catálogo y estados de pantalla	“El listado se lee desde Room. Al abrir la app se intenta actualizar con Retrofit; mientras tanto se puede mostrar carga, error con reintento o el contenido disponible.”
1:30–2:05	Detalle y favoritos	“El detalle solicita información ampliada al segundo endpoint. El botón de favorito actualiza Room y el cambio se refleja en la sección de favoritos.”
2:05–3:00	Modo avión y favorito persistente	“La copia local permite consultar el catálogo sin internet. Si no hay copia, se muestra un error recuperable; los favoritos sobreviven al cierre y a una sincronización.”
3:00–3:50	Cerrar sesión e ingresar como administrador	“La sesión y el rol provienen de Firebase y Firestore. El panel administrativo solo aparece cuando el rol leído es admin.”
3:50–4:30	Sincronización, aviso y caché	“Al sincronizar, el repositorio guarda los resultados en Room. Si el sistema permite avisos, se notifica cuántos elementos se guardaron. El panel puede borrar la caché.”
4:30–5:00	Resumen de arquitectura y pruebas	“Compose observa el estado del ViewModel; el ViewModel usa interfaces de repositorio. Las pruebas unitarias verifican estados y comportamiento con datos falsos.”

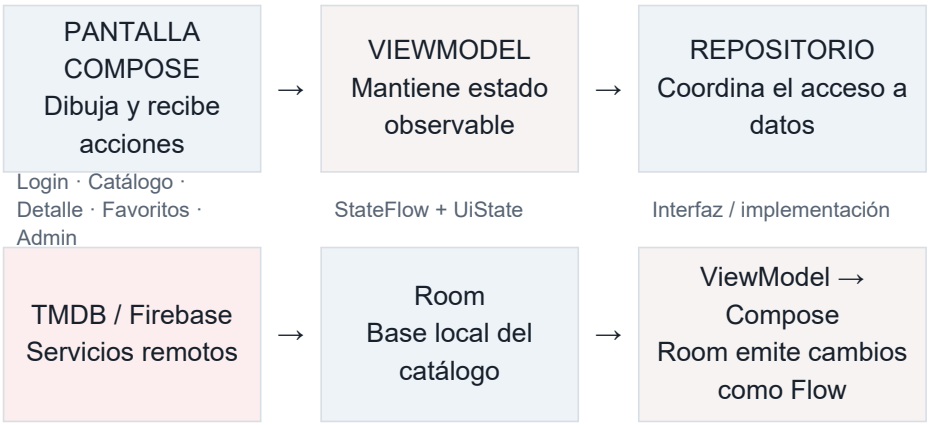
Consejo: antes de empezar, confirma que las cuentas de demostración funcionan, que hay conexión para la primera descarga y que sabes activar el modo avión. Evita mostrar contraseñas o claves.

Frase de apertura sugerida

“Voy a presentar Cine Castillo desde los requisitos del laboratorio: autenticación con roles, consumo de una API de películas, almacenamiento local para favoritos y modo sin conexión, notificaciones, panel de administración y pruebas.”

Cómo funciona la aplicación, de extremo a extremo

Piensa en la app como un equipo con responsabilidades separadas: la pantalla presenta; el ViewModel organiza el estado; el repositorio decide de dónde obtener o guardar información. Así una pantalla no se conecta directamente con Firebase, Retrofit ni Room.



La ruta de autenticación también está separada

Paso	Qué ocurre
1. Firebase Authentication	Verifica las credenciales y mantiene la identidad de la sesión.
2. Firestore: usuarios/{uid}	La app lee el campo rol del documento asociado al usuario autenticado.
3. Navegación por rol	El rol usuario ve funciones de usuario; admin recibe acceso al panel. El rol no se decide por un botón ni se inventa dentro de la pantalla.

Analogía fácil: Firebase comprueba quién eres; Firestore consulta qué permiso tiene esa persona. El repositorio es el encargado de hablar con las fuentes de datos. La pantalla solo presenta el resultado.

Términos que debes poder explicar

• StateFlow: una corriente de información que avisa al ViewModel cuando cambian los datos.

• UiState: una etiqueta clara para saber si la pantalla está cargando, tiene datos o debe mostrar un error.

• Repositorio: una capa intermedia que oculta los detalles de red y base local.

Requisitos esenciales: acceso, roles y catálogo

Req.	Qué pide el laboratorio	Cómo explicarlo y demostrarlo
R01 · Acceso	Correo y contraseña con Firebase Authentication; validar campos vacíos y mostrar errores.	Deja un campo vacío para mostrar validación; luego demuestra credenciales incorrectas con un mensaje legible.
R02 · Roles	Leer admin/usuario desde Firestore y limitar las pantallas según el rol.	Entra con usuario y muestra que no ve Admin. Cierra sesión, entra con admin y muestra el panel. Los documentos de prueba se preparan en Firebase Console.
R03 · Lista	Retrofit para obtener listado; mostrar carga, error con botón Reintentar y estado vacío.	Explica que el catálogo se descarga y luego se consulta desde Room. Si hay error y Room está vacío, aparece reintento.
R04 · Detalle	Una segunda solicitud API obtiene el detalle del elemento seleccionado.	Abre una película y explica que la pantalla pide el detalle por su identificador; no es solo el texto resumido de la tarjeta.
R05 · MVVM	ViewModel, UiState y una interfaz de repositorio.	Señala que Compose llama acciones del ViewModel y observa el estado; el ViewModel depende de contratos, no de una pantalla ni de Firebase directamente.

Explicación sencilla de los tres estados

Estado	Qué ve la persona	Cuándo aparece
Cargando	Indicador de espera.	Se está leyendo o actualizando la información y aún no hay datos locales.
Éxito	Catálogo o resultado disponible.	Room tiene datos para mostrar; pueden actualizarse en segundo plano.
Error	Mensaje y opción de reintentar.	No se pudo obtener datos y tampoco hay copia local suficiente.

Punto que suele preguntar el profesor

¿Por qué no leer Retrofit directamente desde la pantalla? Porque la pantalla quedaría acoplada a la red y sería difícil reutilizarla o probarla. Con MVVM, la pantalla observa un estado; el ViewModel coordina acciones y el repositorio decide la fuente de datos.

Room, favoritos, trabajo sin conexión y notificaciones

• Room: almacena localmente el catálogo y el estado de favorito. Su DAO ofrece consultas observables para lista, detalle y favoritos.

• Fuente local: las pantallas reciben el flujo de Room; cuando se sincronizan datos, la interfaz cambia al observar la base.

• Favoritos: el cambio se escribe en Room y permanece tras cerrar la app. Al sincronizar una película ya guardada, se conserva su marca de favorito.

• Sin internet: si Room tiene datos, la app los muestra e indica que está sin conexión; si no tiene datos, ofrece error recuperable y reintento.

• Notificación local: al finalizar la sincronización informa cuántos elementos se guardaron. Android requiere canal y, en versiones recientes, permiso de notificaciones.

• Panel admin: presenta totales guardados y favoritos, permite sincronizar y vaciar la caché. Se ve solo con rol admin.

• Sesión: Firebase mantiene al usuario autenticado al reabrir; cerrar sesión elimina la sesión y vuelve al acceso.

Fragmento clave: proteger favoritos al actualizar la lista

Código real simplificado de la idea central en el repositorio. Para la exposición, enfócate en las dos cosas que conserva y en el momento en que escribe en Room.

RoomMovieRepository · refrescar catálogo

```
val remote = api.discover().results
database.withTransaction {
    val local = dao.getAll().associateBy { it.id }
    val updated = remote.map { dto ->
        dto.toEntity(
            favorite = local[dto.id]?.favorite ?: false,
            detailLoaded = local[dto.id]?.detailLoaded ?: false
        )
    }
    dao.upsert(updated)
}
```

Línea o idea	Qué significa
api.discover()	Pide el catálogo actualizado al servicio remoto.
withTransaction	Agrupa la escritura para que Room aplique el cambio de forma consistente.
local[dto.id]?.favorite	Busca la misma película en la base local y conserva si era favorita.
dao.upsert(updated)	Guarda o reemplaza las películas; luego Room avisa a los observadores.

En una frase: “La API trae los datos recientes, pero la actualización no borra mis favoritos: el repositorio combina lo remoto con el valor local y guarda el resultado en Room.”

Los 14 casos de aceptación que debes poder demostrar

La guía del laboratorio incluye comprobaciones funcionales de autenticación, red, datos locales, permisos y recuperación. Esta tabla te sirve de lista durante la sustentación; las capturas correspondientes están en la carpeta de evidencias.

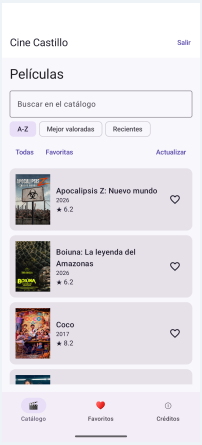
Caso	Qué se demuestra	Qué decir al profesor
PA01	Cuenta usuario entra al catálogo y no ve Admin.	El rol controla las opciones.
PA02	Cuenta admin abre el catálogo y el panel administrativo.	El rol se lee desde Firestore.
PA03	Contraseña incorrecta muestra error claro.	El error no deja la pantalla bloqueada.
PA04	Campos vacíos se validan antes de autenticarse.	Se evita una solicitud inválida.
PA05	Primera descarga: catálogo de 20 y aviso; tocar aviso abre la app.	Retrofit trae el catálogo y la app lo guarda localmente.
PA06	Detalle usa la segunda ruta de TMDB.	El detalle se consulta con el ID de la película.
PA07	Favorito permanece al detener y reabrir.	Room persiste el estado.
PA08	Modo avión con catálogo previo muestra datos locales y aviso offline.	Room permite continuidad sin internet.
PA09	Sin datos y sin internet aparece reintento; al recuperar red se vuelve al acceso.	Sin caché no se finge que existen datos.
PA10	Favorito se conserva después de sincronizar.	La actualización combina datos locales y remotos.
PA11	Vaciar caché deja cero; al reintentar se recupera catálogo y totales.	El panel demuestra limpieza y recuperación.
PA12	Cerrar sesión y reabrir obliga a autenticarse.	Se invalida la sesión local de Firebase.
PA13	Giro conserva filtro y no descarga de nuevo.	El estado del ViewModel no depende de recrear la pantalla.
PA14	Denegar permiso no detiene la sincronización ni el catálogo.	La notificación es opcional; el resto funciona.

Evidencia que conviene tener abierta

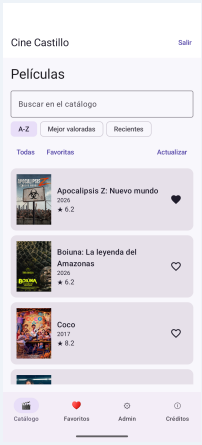
Abre la matriz de aceptación y las capturas PA01–PA14. Para una demo en vivo, los casos más informativos son PA01/PA02 (roles), PA05/PA06 (API y detalle), PA07/PA08 (persistencia y offline), PA11 (panel y recuperación) y PA14 (permiso denegado). No dependas de internet para demostrar PA08; activa modo avión después de haber descargado el catálogo.

Capturas de referencia para la demostración

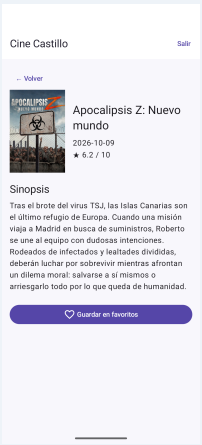
Estas evidencias pertenecen a los escenarios del laboratorio. Úsalas para localizar rápidamente qué estado debes abrir si el emulador, la conexión o el tiempo no permiten repetir cada paso en vivo.



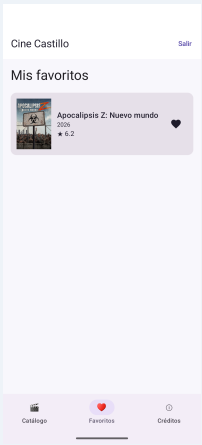
Usuario: catálogo sin acceso Admin



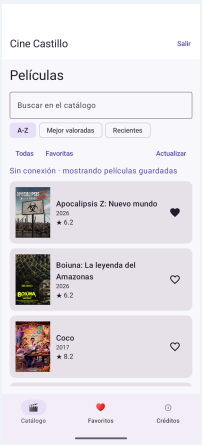
Administrador: panel y opciones



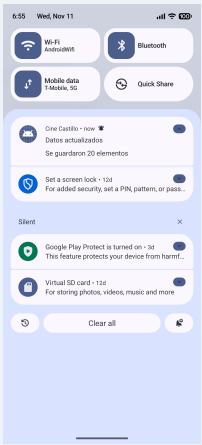
Detalle ampliado de la película



Favorito conservado en Room



Catálogo local en modo avión



Sincronización y notificación

Las capturas complementan la demostración. Si las pruebas cambian por el estado actual de la cuenta, usa el registro documentado como respaldo y explica las condiciones: red, caché y permiso del sistema.

Preguntas probables y respuestas que debes comprender

¿Por qué existe una interfaz MovieRepository?

Para separar la lógica que usa el ViewModel de la tecnología concreta. El ViewModel puede trabajar con una implementación real o falsa en una prueba.

¿Por qué la interfaz lee Room y no directamente TMDB?

Room funciona como fuente local de la pantalla. Así el catálogo puede aparecer rápido y seguir disponible sin conexión; Retrofit actualiza Room cuando hay internet.

¿Qué pasa con favoritos al sincronizar?

El repositorio busca cada película en Room, conserva su marca favorita y luego guarda la versión combinada.

¿Dónde se decide si alguien es admin?

Después de autenticar, se consulta el documento usuarios/{uid} en Firestore y su campo rol. La pantalla recibe ese rol y la navegación limita el panel.

¿Qué pasa si se deniega el permiso de notificaciones?

La app sigue sincronizando y mostrando el catálogo; solo se omite el aviso del sistema.

Pruebas y rúbrica

• El registro del proyecto documenta 14 casos de aceptación aprobados en Pixel 8 y 8 pruebas unitarias. Las unitarias incluyen siete comprobaciones del ViewModel de catálogo y una de autenticación.

• La rúbrica asigna 20 puntos: MVVM 5; Retrofit 3; Room y modo offline 3; Firebase, roles y sesión 3; Compose y notificaciones 2; Git/GitHub 3; mejora asistida 1.

• La metodología distingue main (desarrollo base) de mejora-ia. La guía pide al menos 15 commits en main, un mínimo de 6 por integrante, al menos 3 en mejora-ia, PROMPTS.md, una solicitud de integración y etiqueta v1.0.

Cierre sugerido

“El proyecto demuestra cómo integrar autenticación por roles, consumo de datos remotos, persistencia local y una interfaz reactiva. Room permite mantener el catálogo y los favoritos disponibles sin conexión; las pruebas verifican tanto el flujo normal como los casos de error y recuperación.”

Antes de la sustentación

• Ten a mano la app, las cuentas de prueba y la matriz con capturas; no muestres contraseñas ni claves.

• Practica explicar con tus palabras repositorio, Room como fuente local, rol desde Firestore y permiso de notificaciones.

• Si una prueba depende de internet, advierte cuándo estás desconectando y reconectando la red.

Base de esta guía: indicaciones del laboratorio, estructura y README del proyecto, y registro PRUEBAS_ACEPTACION.md. Se limita a los requisitos originales del docente.